

contact details

hyder: syedhyder@ineuron.ai
nitin : nitin@ineuron.ai

What will be the result of compiling and executing Test class?

```
public class Test {  
    public static void main(String[] args) {  
        byte var = 100;  
        switch(var) {  
            case 100:  
                System.out.println("var is 100");  
                break;  
            case 200:  
                System.out.println("var is 200");  
                break;  
            default:  
                System.out.println("In default");  
        }  
    }  
}
```

- A. var is 100
- B. var is 200
- C. In default
- D. CompileTime error.

Answer: D

What will be the result of compiling and executing Test class?

```
public class Test {  
    public static void main(String[] args) {  
        String fruit = new String(new char[] {'M', 'a', 'n', 'g', 'o'});  
        switch (fruit) {  
            default:  
                System.out.println("ANY FRUIT WILL DO");  
            case "Apple":  
                System.out.println("APPLE");  
            case "Mango":  
                System.out.println("MANGO");  
            case "Banana":  
                System.out.println("BANANA");  
                break;  
        }  
    }  
}
```

- A. ANY FRUIT WILL DO
- B. MANGO
- C. MANGO
BANANA
- D. ANY FRUIT WILL DO
APPLE
MANGO
BANANA

Answer: C

Q>

Consider below code:

```
//Test.java
public class Test {
    public static void main(String [] args) {
        boolean flag = !true;//false
        System.out.println(!flag ? args[0] : args[1]);
    }
}
```

What will be the result of compiling and executing Test class using below commands?

```
javac Test.java
java Test AM PM
```

- A. AM
- B. PM
- C. ExceptionInitializerError while loading the .class file
- D. CompilationError

```
args[0] = "AM"
args[1] = "PM"
System.out.println(true? "AM":"PM");
```

Answer: A

What will be the result of compiling and executing Test class?

```
public class Test {
    public static void main(String [] args) {
        int a = 3;// a = 4
        System.out.println(a++ == 3 || --a == 3 && --a == 3);//true
        System.out.println(a);//4
    }
}
```

- A. true
3
- B. false
3
- C. True
3
- D. False
3
- E. true
4
- F. false
3
- G. CompilationError

Answer: E

Q>

What will be the result of compiling and executing Test class?

```
public class Test {
    public static void main(String [] args) {
        int a = 3;
        m(++a, a++);
        System.out.println(a);
    }
    private static void m(int i, int j) {
        i++;
        j--;
    }
}
```

- A. 4
- B. 5
- C. 6
- D. 3

Answer: B

Q>

Consider below code of Test.java file:

```
public class Test {  
    public static void main(String [] args) {  
        boolean flag = false; //true, false  
        System.out.println((flag = true) | (flag = false) || (flag = true)); //  
true | false => true || (flag=true)  
        System.out.println(flag); //false  
    }  
}
```

What is the result of compiling and executing Test class?

- A. true
false
- B. false
true
- C. true
true
- D. false
false
- E. CompilationError

Answer: A

Q>

Consider below code of Test.java file:

```
public class Test {  
    public static void main(String [] args) {  
        boolean status = true;  
        System.out.println(status = false || status = true | status = false);  
        System.out.println(status);  
    }  
}
```

What is the result of compiling and executing Test class?

- A. true
false
- B. false
true
- C. true
true
- D. false
false
- E. CompilationError

Answer: E

Q>

What will be the result of compiling and executing Test class?

```
public class Test {  
    public static void main(String[] args) {  
        String msg = "Hello";  
        boolean [] flag = new boolean[1];
```

```

        if(flag[0]) {
            msg = "Welcome";
        }
        System.out.println(msg);
    }
}

```

- A. Hello
- B. Welcome
- C. ArrayIndexOutOfBoundsException
- D. NullPointerException
- E. CompileTimeError

Answer: A

```

public class Test {
    public static void main(String [] args) {
        boolean flag1 = true;
        boolean flag2 = false;
        boolean flag3 = true;
        boolean flag4 = false;
        System.out.println(!flag1 == flag2 != flag3 == !flag4); //Line n1
        System.out.println(flag1 = flag2 != flag3 == !flag4); //Line n2
    }
}

```

What will be the result of compiling and executing Test class?

- A. Line n1 cause compilation error
- B. Line n2 causes compilation error
- C. true
true
- D. true
false
- E. false
true
- F. false
false

JVM

===

```

!true == false != true == !false
false == false != true == !false
    true  != true == !false
        false == true
            false

```

```

flag1 = false != true == !false
flag1 = true == true
flag1 = true
    true

```

Answer: E

Q>

```

public class Test {
    public static void main(String[] args) {
        int score = 30; // Line n1
        char grade = 'F'; // Line n2
        if (50 <= score < 60) // Line n3
            grade = 'D';
    }
}

```

```

        else if (60 <= score < 70) // Line n4
            grade = 'C';
        else if (70 <= score < 80) // Line n5
            grade = 'B';
        else if (score >= 80)
            grade = 'A';
        System.out.println(grade);
    }
}

```

- A. Compilation error
- B. A
- C. B
- D. C
- D. F

System.out.println(10<20<30); //CE:nesting or relational operator is not possible

Answer: A

Q>

Consider below code of Test.java file:

```

public class Test {
    public static void main(String[] args) {
        int x = 10; //Line n1
        if (false)
            System.out.println(x); //Line n2
        System.out.println("HELLO"); //Line n3
    }
}

```

What is the result of compiling and executing Test class?

- A. Compilation error at Line n1
- B. Compilation error at Line n2
- C. Compilation error at Line n3
- D. HELLO
- E. 10
HELLO

Answer: D(unreachable won't be checked by compiler for if and else logic)

game of static

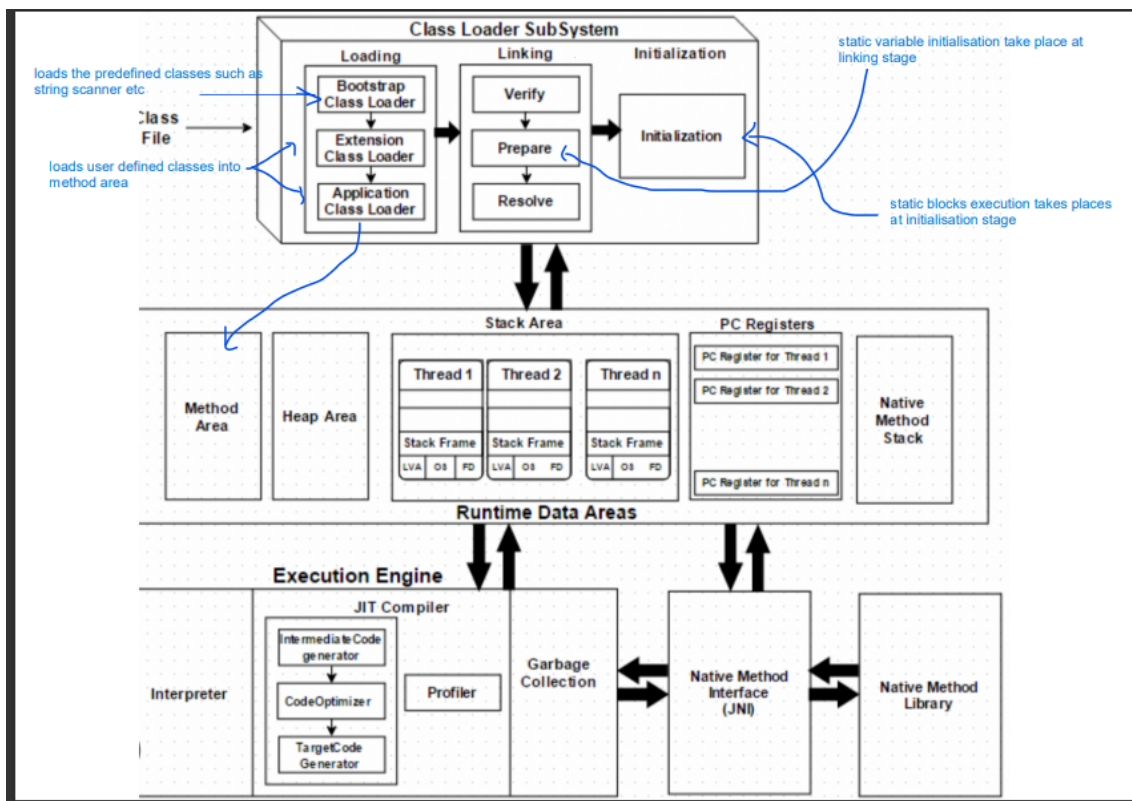
code execution in java at jvm level

- static variables
- static blocks
- static methods(only psvm called automatically)
- instance variables initialisation
- non static block
- constructor
- class methods

you can not fetch a non static method from a static method while in a non static method a static method can be called this is because at the class loader sub system a non static method does not get created however when a static method is called in non static method or block already the static method created first so it will be fetched

static variable property

- ⇒ ① static keyword used
- ② class loading
- ③ Heap
- ④ memory allocated
- only one in heap
- ⑤ one copy static var used by all the object
- ⑥ using class Name
- ⑦ class variables
- ⑧ object independent
- ⑨ around inside static element non static
- ⑩ = 5



```

class Demo1
{
    int a;
    int b;

    static int count;

    {
        count++;

        System.out.println("Hello");
    }

    Demo1()
    {
        this(100);
        System.out.println("First con");
    }

    Demo1(int a)
    {
        System.out.println(a);
        this.a=a;
    }

    Demo1(int a, int b)
    {
        this.a=a;
        this.b=b;
    }
}

class LaunchNonStaticBlock {

    public static void main(String[] args) {
        Demo1 d1=new Demo1();
        Demo1 d2=new Demo1(10,20);
        Demo1 d3=new Demo1(10);

        System.out.println(Demo1.count);

    }

}

```

- output is

- Hello
100
First con
Hello
Hello
10
3
Process finished with exit code 0